

1. JWT vs Devise

JWT (JSON Web Token) and **Devise** are both used for authentication, but they work very differently.

- **Devise** is a full authentication library for Rails that offers ready-made user signup, login, password reset, session management, email confirmation, and more. It normally uses **sessions (cookies)** to keep a user logged in. It manages everything for you out of the box and integrates well with Rails controllers and views.
- **JWT** is a **stateless token-based authentication** system, where the server issues a signed token after login, and the client sends that token in the Authorization header with future requests. The server can verify the token without storing session data. This makes JWT suitable for **REST APIs**, mobile apps, and decoupled frontends.

Key differences:

- Devise keeps session state on the server and uses cookies, while JWT keeps authentication info inside the token itself (no server session needed).
 - JWT scales better for APIs and distributed systems, but token revocation is harder unless you implement a blacklist. Devise makes session expiration and logout easier.
-

2. REST API vs GraphQL

REST and **GraphQL** are both ways for clients and servers to communicate, but they differ in style and flexibility.

- **REST API** is an architectural style that uses multiple endpoints (URLs) for different resources (e.g., /users, /posts) and HTTP verbs like GET, POST, PUT, DELETE to fetch or change data. Each endpoint returns a fixed data structure defined by the server.
- **GraphQL** is a **query language and API runtime** that uses a **single endpoint** (e.g., /graphql) where the client defines exactly what data it needs through queries and mutations. This allows clients to fetch nested data in one request and only request the fields they care about.

Key differences:

- REST has many endpoints and fixed responses decided by the server, while GraphQL has one endpoint and flexible responses defined by the client.

- REST can suffer from over-fetching (getting more data than needed) or under-fetching (needing multiple requests), whereas GraphQL solves this by letting the client specify required fields.
 - REST versioning often uses URL versioning (e.g., /api/v1/), while GraphQL avoids versioning by evolving schemas without breaking clients.
-

3. Session vs Cookies in Rails

In Rails, **sessions** and **cookies** are related but different.

- A **cookie** is a small key-value stored by the browser and sent with every request to the server. It can hold data directly or an identifier for a session.
- A **session** is server-side storage that holds data about a logged-in user (like user ID) between requests, and usually relies on a cookie to store the session ID.

How they work:

- In **session-cookie authentication**, after login, Rails stores a session ID in a cookie on the browser, and on every request the server looks up the session ID to know which user is logged in. The actual session data remains secure on the server.
 - Cookies can also store authentication tokens (like JWT), but sessions are about maintaining state on the server, making cookie-based sessions secure and easy for full-stack apps.
-

4. Pundit vs CanCanCan

Both **Pundit** and **CanCanCan** are authorization libraries in Rails used to control who can do what in your application.

- **Pundit** uses **policy classes** where each resource has a policy with methods defining what actions a user is allowed to take. This gives clear, explicit authorization logic and strong separation of concerns.
- **CanCanCan** uses a central **ability file** where you define all permissions in one place using a DSL (can :read, Post). It's simpler for small apps but can grow complex as rules increase.

Comparison:

- Pundit's policies are simpler to test and reason about since each resource's rules are in one file.
 - CanCanCan's abilities make it easy to write rules quickly but can become hard to manage with many resources or complex rules.
-

5. How Devise works and its configuration in Rails

Devise is a popular authentication gem for Rails that provides ready-made solutions for signing up, signing in, logging out, password resets, confirmations, and more. It is built on the Warden authentication framework and integrates tightly with Rails.

How it works:

1. Add Devise to your Gemfile and run `bundle install`.
2. Run `rails generate devise:install` to set up initial configuration.
3. Create a Devise user model with `rails generate devise User`.
4. Run migrations (`rails db:migrate`) to create the users table with authentication fields.
5. Devise adds routes and helpers like `authenticate_user!` to protect endpoints.
6. On login, Devise stores the user session so future requests are authenticated automatically.

Devise automatically manages session cookies, encryption, and sign-in/out flows with minimal manual setup.

6. How JWT works and its configuration in Rails

JWT (JSON Web Token) works by encoding user information and expiration into a **signed token** that the server issues after a successful login. The client sends this token with every request (usually in `Authorization: Bearer <token>` header), and the server verifies it on each request without storing session data.

How it works step-by-step:

1. Client sends credentials (email/password) to the login endpoint.
2. Server verifies credentials and returns a signed JWT token with payload (like user ID and expiration time).

3. Client stores the token (in localStorage, sessionStorage, or cookie).
4. For each API request, the client includes the token in the Authorization header.
5. Server decodes and verifies the token signature and expiration to authenticate the request.

Configuration in Rails:

- Add the jwt gem and optionally devise-jwt if using Devise.
 - Set up token secret in credentials/environment.
 - Create login controller to issue tokens.
 - Decode and verify tokens in application controller before protected actions.
-

7. How Pundit works and its configuration

Pundit works by defining **policy classes** where each class maps to a model and contains methods for each action (like show?, update?). These methods take the current user and the record, allowing you to decide if the user has permission.

Configuration in Rails:

1. Add gem 'pundit' to Gemfile and bundle install.
 2. Run rails generate pundit:install to create the application policy file.
 3. Generate policy for a resource (rails generate pundit:policy Post).
 4. In controllers, call authorize @post before actions.
 5. Define logic inside policy methods (e.g., only owner can edit).
-

8. How CanCanCan works and its configuration

CanCanCan works by defining a central **Ability** class where you specify what the current user **can** and **cannot** do with different models using simple DSL rules.

Configuration in Rails:

1. Add gem 'cancancan' to Gemfile and bundle install.
 2. Run rails generate cancan:ability to create ability.rb.
 3. In ability.rb, define rules like can :manage, Post, user_id: user.id.
 4. In controllers, use authorize! :update, @post to enforce permissions.
 5. CanCanCan automatically raises exceptions when unauthorized.
-

9. How REST API works and how to create it in Rails

A **REST API** in Rails uses controllers and routes that respond with JSON instead of HTML. Each resource has its own set of routes and actions corresponding to HTTP verbs (GET, POST, PATCH/PUT, DELETE).

How it works:

- Rails routes like resources :users generate RESTful routes.
- Controllers respond with JSON (render json: @user).
- Clients include tokens or session cookies for authentication.
- The API is stateless: each request contains all needed info.

How to create:

1. Create Rails app (rails new my_api --api).
 2. Generate controllers and models.
 3. Set routes.
 4. Use serializers or JSON builders.
 5. Protect routes with authentication.
-

10. How GraphQL works and how to configure it in Rails

GraphQL is a query language and runtime where clients send a structured query to a single endpoint and specify exactly what data they want back. GraphQL resolves queries based on a defined **schema** with types and fields.

How it works:

- You define a GraphQL schema with types and fields.

- Clients send queries for specific fields (like user name, posts).
- Server executes queries and returns exactly the requested data.
- Mutations allow updating data.

How to configure in Rails:

1. Add graphql gem and run generator (`rails generate graphql:install`).
2. Define types, queries, and mutations in the schema folder.
3. Set up a route (usually `/graphql`).
4. Use resolvers to fetch data from models.
5. Secure endpoints with authentication checks.